

RWMS — Система управления складскими и арендными операциями

Презентационный документ о продукте: проблематика, ценность, домен, архитектура и перспективы

Дата: октябрь 2026 · Проект: RWMS (Rental Warehouse Management System)

Содержание

1. Резюме для руководства
 2. Бизнес-контекст и проблематика
 3. Решение и ценностное предложение
 4. Пользователи и клиентские поверхности
 5. Доменная модель: полный жизненный цикл бытовки
 6. Архитектура платформы
 7. Безопасность, надёжность и качество
 8. Инженерная культура и доказательность
 9. Текущее состояние и вехи
 10. Дорожная карта и перспективы
 11. Риски и их митигация
 12. Заключение
 13. Приложение А: каталог сервисов
 14. Приложение В: источники истины
-

1. Резюме для руководства

RWMS — действующая рабочая система, которая объединяет разрозненные операции по учёту, аренде, ремонту и перемещению модульных зданий (бытовок, кабин, оборудования) в один цифровой контур: **веб-панель для менеджеров и четыре мобильных Android-приложения** для менеджера, работника, водителя и клиента.

Ключевые факты:

- **12 независимо владеющих данными доменных сервисов** + stateless API gateway; каждый бизнес-инвариант и статус имеет ровно одного владельца.
- **Сквозной жизненный цикл актива**: регистрация → бронь → аренда → возврат → осмотр → смета → ремонт → приёмка → перемещение/списание — полностью оцифрован.

- **Операции в поле:** задания, фотофиксация, хронометраж, совместная работа стропальщиков, офлайн-синхронизация мобильных клиентов.
- **Логистический планировщик** с картографией, изохронами, слотами доставки и сагами отгрузок/перемещений.
- **Доказуемое качество:** канонические OpenAPI- и event-контракты, архитектурные policy-тесты, регулярные полные аудиты кодовой базы (последний: 5 421 файл, 1,36 млн строк, 135 карточек проблем с журналом исправлений).

Ценность для бизнеса — сокращение ручного учёта и потерь активов, прозрачность загрузки парка, управляемый ремонт с бюджетом и приёмкой, единая версия истины для склада, офиса и поля.

2. Бизнес-контекст и проблематика

2.1 Отраслевой контекст

Компании, сдающие в аренду модульные здания, бытовки и строительное оборудование, управляют парком из десятков и сотен единиц, распределённых по нескольким складам. Каждый актив проходит длинный цикл: покупка/изготовление, простой, бронирование под клиента, отгрузка, аренда, возврат с повреждениями, ремонт, повторная выдача, перемещение между складами, в конце — продажа или списание.

2.2 Типичные боли отрасли (без системы)

#	Проблема	Последствие для бизнеса
1	Учёт в таблицах, тетрадях и «в голове»	Расхождение фактических остатков с документами; потерянные активы
2	Нет единого статуса доступности	Срыв сроков отгрузки, двойные брони, простой парка
3	Повреждения при возврате не фиксируются системно	Неустранённый ремонт выдаётся следующему клиенту; репутационные потери
4	Ремонт без смет, планов и приёмки	Непрозрачные затраты, переделки, споры с подрядчиками
5	Логистика по телефону «кто свободен»	Пустые рейсы, невозможность обещать клиенту точный слот доставки
6	Полевые задачи без подтверждения	Нет хронометража, нет фото-доказательств, непонятно, кто и что сделал
7	Инвентаризация — болезненный ручной срез	Ревизия занимает дни, замораживает работу склада

#	Проблема	Последствие для бизнеса
8	Нет аналитики загрузки и оборачиваемости	Управленческие решения вслепую: что докупать, что продавать

2.3 Почему это сложно автоматизировать «обычным CRUD»

- Статус актива определяется **одновременно** наличием брони, lease'a, незавершённого ремонта, перемещения и инвентаризации — нужен единственный владелец переходов, иначе статусы расходятся между таблицами и экранами.
- Деньги, время и разрушительные операции требуют **идемпотентности, версий и аудита**, а не «последнего побеждающего» обновления.
- Полевые клиенты работают в сетях низкого качества — нужна офлайн-устойчивость и надёжное восстановление (загрузки медиа, назначения, рейсы).
- Междоменные процессы (отгрузка = резерв + заказ + транспорт + задача работнику) нельзя вести в одной транзакции БД — нужны саги и событийная согласованность.

Именно эти ограничения сформировали облик RWMS: микросервисная платформа с жёсткими границами владения вместо монолита, где «всё зависит от всего».

3. Решение и ценностное предложение

3.1 Elevator pitch

RWMS — единая система, в которой каждая бытовка всегда знает, где она, в каком состоянии, кому забронирована, сколько стоит её ремонт и когда вернётся в строй. Офис видит это в веб-панели, склад — в приложениях работников и водителей, логист — на карте со слотами доставки.

3.2 Ценность по направлениям

Для парка активов (деньги)

- Единый lifecycle-статус (FREE / BOOKED / RENTED / AFTER_RENT / REPAIR / IN_TRANSFER / SALE / WRITTEN_OFF / LOST...) — парк виден целиком, без «серых зон».
- Holds и резервы гарантируют: забронированное не уйдёт в другую сделку.
- Списания и находки проходят через утверждаемые процедуры с историей решений.

Для аренды и продаж (выручка)

- Запрос клиента → презентация → бронь → заказ → отгрузка с точным слотом.
- Возврат с осмотром: повреждение автоматически порождает черновик сметы, ремонт и приёмку — актив возвращается в строй управляемо.
- Аналитика KPI и загрузок (read-проекция analytics-service) — основа для решений о пополнении/выбытии парка.

Для склада и ремонта (себестоимость)

- Task board: очереди, назначения, активная совместная работа (стропальщик), общее завершение задач, съёмка медиа прямо в задании.
- Каталог ремонта, сметы, этапы, приёмка с правом переделки и предложением о выбытии — затраты под контролем и с доказательствами.
- Инвентаризация: замороженный план, находки, сверка, публикация — ревизия становится процессом, а не авралом.

Для логистики (скорость)

- Python-планировщик с PostGIS и Valhalla: маршруты, изохроны, слоты доставки, режимы работы складов; саги отгрузок/перемещений с восстановлением после сбоев.
- Журнал диспетчера: неизменяемые решения логиста + системные события в одном хронологическом контуре.

Для доверия и контроля (риск)

- OIDC/OAuth2 + PKCE, JWT с edge-политикой на gateway, ролям и складскому admission-контролю.
- Transactional outbox/inbox, идемпотентность, optimistic concurrency, version-fenced эффекты — ни один шаг процесса не теряется и не дублируется.
- Media-service хранит приватные оригиналы и производные в MinIO с proof'ами владельца — фотофиксация как юридическое доказательство состояния актива.

3.3 Дифференциаторы

1. **Один владелец на инвариант** — статус бытовки живёт в asset-service, заказ — в logistics, смета/ремонт — в maintenance; конфликтующих источников истины физически не может возникнуть.
2. **Контракты как закон** — канонические OpenAPI и event-схемы в репозитории; CI-проверки совместимости producer/consumer, byte-identical сгенерированные типы клиентов.
3. **Мобильные клиенты первого класса** — пять отдельных Android-приложений с офлайн-синхронизацией (Room/WorkManager) и публикуемыми неизменяемыми APK.
4. **Инженерная отчётность** — каждый релиз сопровождается gate-прогонами тестов, хешами артефактов, дампами БД и manifest'ами отката (см. раздел 8).

4. Пользователи и клиентские поверхности

4.1 Роли

Роль	Где работает	Основные сценарии
Оператор / менеджер склада	Web-панель (panel)	Парк, заказы, брони, осмотры, сметы, приёмка, инвентаризация
Администратор платформы	Web-панель (admin entripoint)	Пользователи, роли, склады, конфигурация доступа
Менеджер аренды	Android ManagerApp	Клиенты, запросы, заказы, контроль исполнения
Работник склада	Android WorkerApp	Назначения, выполнение задач, совместная работа, фото/медиа
Водитель	Android DriverApp	Рейсы и складские задания водителя, подтверждение отгрузок
Клиент	Android ClientApp	Заявка на аренду, статус заказа
Логист	Logistics-panel (web, карта)	Планирование маршрутов, слоты, распределение доставок, журнал

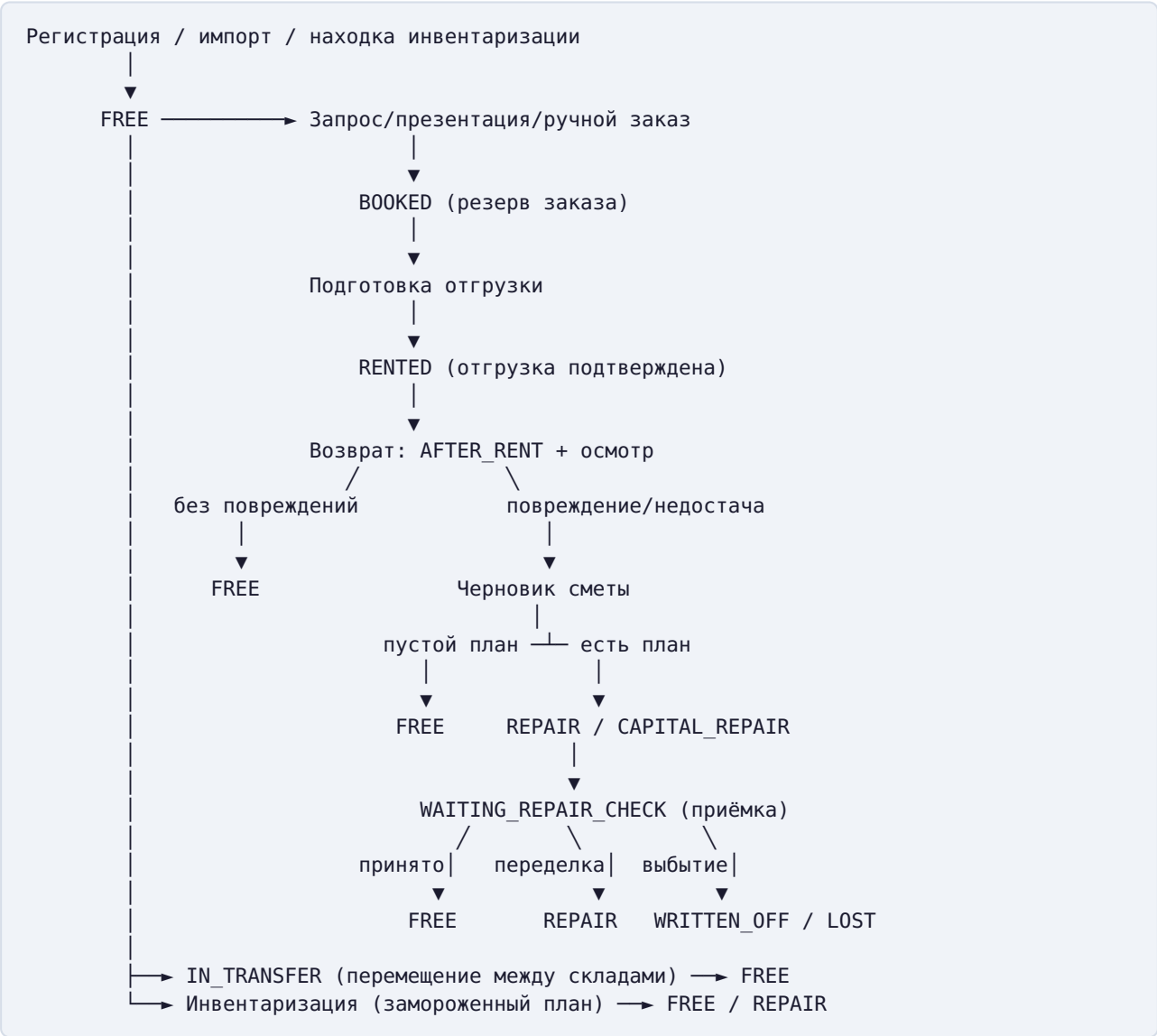
4.2 Клиентские поверхности

- **Web-панель** — React 19 + TypeScript + Vite, TanStack Query/Table/Virtual, Tailwind 4, shadcn/Base UI; отдельные точки входа main/admin/manager.
- **Android-приложения** — Kotlin/Jetpack Compose, OIDC PKCE, Retrofit/Moshi; Worker и Driver — Room + WorkManager для офлайн-работы и надёжной синхронизации.
- **Logistics-panel** — собственный web-клиент с картой (React 19, TS, OIDC, генерируемые из OpenAPI типы).
- **CAD-редактор кабины** (cabin-cad) — Three.js/R3F + Konva + OCCT: паспорт бытовки с геометрией и NDJSON-диагностикой.
- **Download-sites** — публичные страницы релизов трёх APK с проверкой целостности и trust-скриптами публикации.

Все клиенты обращаются только к единому публичному API gateway; браузер никогда не координирует согласованность между сервисами.

5. Доменная модель: полный жизненный цикл бытовки

Центральный процесс продукта — цикл одной единицы актива (RentalItem), подтверждённая карта которого ведётся в базе знаний проекта.



Владелец каждого шага строго определён:

Предмет	Владелец
Идентичность, статус, доступность, остатки, holds, leases	asset-service
Запросы, презентация, бронь, заказ, отгрузка, возврат, перемещение	logistics-service
Каталог ремонта, сметы, план/исполнение ремонта, приёмка, выбытие	maintenance-service
Очереди, назначения, выполнение, хронометраж	task-board-service
Сессии инвентаризации, находки, публикация	inventory-service

Предмет	Владелец
Фото, видео, производные, приватный доступ	media-service
Идентичность и lifecycle склада, admission	warehouse-service

Междоменные переходы пересекают границу владельца **только** через канонический идемпотентный, version-fenced HTTP-эффект либо версионированный событийный факт. У документов, смет, ремонтов, задач и сессий есть собственные локальные циклы — они не подменяют lifecycle актива.

6. Архитектура платформы

6.1 High-level схема



6.2 Принципы, которые держат систему

1. **Один бизнес-инвариант — один сервис-владелец.** Никаких общих таблиц и сквозных Java-моделей домена между сервисами.
2. **Каждый stateful-сервис владеет одной PostgreSQL** и своей историей миграций Flyway (или Alembic для Python-планировщика).
3. **Междоменные связи — версионированные контракты:** OpenAPI для HTTP, event envelopes/AsyncAPI для Kafka. Совместимость проверяется автоматически.
4. **Gateway — транспорт и безопасность, но не оркестратор.** Межсервисный workflow хранится иницилирующим доменом как saga или reconciliation work.
5. **Без 2PC.** Согласованность — через transactional outbox, inbox-дедупликацию, проверку порядка агрегата и версий, идемпотентные повторные попытки.
6. **SSE — сигналы invalidation,** а не канал консистентности; клиенты обновляют только затронутый кэш. Kafka не является базой данных.

6.3 Технологический стек

Область	Технологии
Сервисы	Java 25, Spring Boot 4.1, Gradle (Kotlin DSL), JPA, PostgreSQL, Flyway, Kafka, OAuth2/JWT
Медиа-сервис	Go 1.25, pgx, franz-go, MinIO
Логистический планировщик	Python ≥ 3.12 , FastAPI, async SQLAlchemy, Alembic, PostGIS, Valhalla routing
Web-клиенты	React 19, TypeScript, Vite 7-8, TanStack Query/Table/Virtual, Tailwind 4
Mobile	Kotlin, Jetpack Compose, OIDC PKCE, Retrofit/Moshi, Room, WorkManager
Платформа	platform/: framework-neutral контракты, общий Spring-starter, архитектурные policy-тесты

6.4 Поток запроса (runtime)

1. Клиент получает OIDC-сессию (Authorization Code + PKCE) и ходит только в gateway.
2. Gateway проверяет маршрут и Bearer-токен, чистит forwarding-metadata, передаёт запрос одному сервису-владельцу.
3. Владелец повторно валидирует токен как resource server, применяет складскую и доменную авторизацию, проверяет optimistic concurrency/idempotency, фиксирует свою транзакцию.
4. Нужный другим доменам факт та же транзакция пишет в outbox; relay публикует в Kafka.
5. Consumer дедуплицирует через inbox, проверяет порядок и версию, меняет только своё состояние или read-проекцию.

7. Безопасность, надёжность и качество

Безопасность

- OIDC/OAuth2 как единая граница идентичности (auth-service: пользователи, работники, роли, клиенты, доступ к складам).
- Edge-политика JWT на gateway + повторная проверка токена каждым владельцем.
- Направленный admission-контроль складов; CSRF-защита формы входа; приватные медиаобъекты с proof'ами владельца.
- Выявленные в аудитах конфигурационные риски (например, фиксированный credential service-клиента) ведутся как карточки с приоритетом и закрытием.

Надёжность

- Идемпотентные команды с version fences; безопасные повторы quarantine-saga.
- Саги с immutable receipt: после owner-commit восстановление движется только вперёд.
- Мобильные клиенты: надёжное восстановление загрузок, офлайн-очереди.
- Публикации: pre-deploy дампы PostgreSQL с проверкой pg_restore --list, rollback-образы, manifest'ы точных SHA-256 source/artifact hashes.

Качество

- Policy-gates в CI: полный прогон тестов standalone-backend (546 passed), frontend (274/274), typecheck/lint/build, Ruff + strict мургу, Alembic clean/upgrade/check, byte-identical OpenAPI, Spring-наборы 960/960.
- Архитектурные тесты platform'a запрещают нарушение границ владения.
- Независимые финальные gate-прогоны перед публикацией.

8. Инженерная культура и доказательность

Отличительная черта проекта — **каждое утверждение подкреплено артефактом**:

- **Полный последовательный аудит** кодовой базы: 5 421 versioned файл, 1 361 572 строки; 135 уникальных проблем (1 High / 38 Medium / 96 Low) — каждая как карточка с доказательством и рекомендацией.
- **Журнал проблем и исправлений** (problem.md): непрерывный реестр; новые дефекты сначала фиксируются здесь, исправления не удаляются из истории.
- **Журнал публикаций**: дата, статус, адрес, хеши контейнеров и бандлов, результаты всех gate-прогонов, список дампов и rollback-точек.
- **База знаний** docs/project-knowledge/: поддерживаемая карта архитектуры, runtime-поток, доменной логики и полного цикла бытовки — с датой подтверждения и ссылкой на владельцев.
- **Иерархия источников истины**: продуктовая задача → канонические контракты → код/миграции/тесты владельца → база знаний → исторические материалы (только как свидетельства).

Эта культура снижает главный риск растущей платформы: «система, которую никто не понимает», заменяется системой, которую можно проверить по документам и тестам.

9. Текущее состояние и вехи

Контейнер разработки уже переведён в рабочий режим: система опубликована и доступна по рабочему адресу (logistics-panel, панель WMS, страница загрузок APK), контейнеры healthy без рестартов, миграции применены до текущих heads.

Хронология ключевых вех (по материалам репозитория):

Период	Веха
2026-08	Формализованы границы владения; полный архитектурный аудит; карта цикла бытовки подтверждена
2026-08-09	Пакеты исправлений operator-дефектов логистики; журнал диспетчера; точный перенос доставки по рассчитанным слотам
2026-09	Защита повторяющихся саг (owner commit + immutable receipt); серия полных gate-публикаций на VPS
2026-09	Секвенциальный аудит всей базы (135 карточек); приоритет R-095 (auth-конфигурация) взят в работу
далее	Развитие CAD-паспортов кабин, клиентского Android, ассистента и аналитических проекций

Готовность по областям: ядро цикла (asset/logistics/maintenance/task-board) — рабочее; логистическое планирование — рабочее и активно развиваемое; мобильные клиенты — опубликованы; read-модели (dossier/analytics/assistant) — действуют как проекции поверх доменов.

10. Дорожная карта и перспективы

Ближайшее (закрепление результата)

- Заккрытие High-приоритета аудита (конфигурация auth-установки) и пакета Medium.
- Стабилизация release-процесса: воспроизводимые деплои из main-ветки, расширение trust-проверок APK-публикаций.

Среднесрочное (расширение ценности)

- Углубление клиентского портала (ClientApp): самостоятельные заявки, статусы, документы — снижение нагрузки на менеджеров.
- Аналитика: KPI загрузки парка, оборачиваемости, стоимости цикла «возврат → ремонт → выдача»; прогнозирование простоев.
- Ассистент: диалоговые сценарии поверх доменных инструментов (аренда остаётся во владении logistics).
- CAD-паспорт кабины как стандарт комплектации и основания смет.

Долгосрочное (масштабирование)

- Мультикомпанийность и тарификация (SaaS-модель для других парков аренды).
- Расширение типов активов (оборудование, техника) на тех же границах владения.
- Интеграции с внешними системами (1C/ERP, ЭДО, телеметрия) через контракты, не нарушая владельцев.

11. Риски и их митигация

Риск	Вероятность/ влияние	Митигация в проекте
Расползание границ при росте команды	Среднее/ Высокое	Архитектурные policy-тесты, контрактные проверки, база знаний с владельцами
Конфигурационные security-дефекты среды	Низкое/Высокое	Реестр проблем с приоритетами (R-095 первым), отдельные аудиты auth-контура
Операторские ошибки в поле	Среднее/ Среднее	Идемпотентность, version fences, неизменяемые решения с журналом, offline-восстановление
Потеря данных при деплое	Низкое/Очень высокое	Обязательные pre-deploy дампы с проверкой, manifest'ы хешей, rollback-образы
Технический долг (96 Low-карточек)	Высокое/Низкое	Непрерывный журнал: дефекты не удаляются, исправления фиксируются прогонами

12. Заключение

RWMS решает не «задачу учёта бытовок», а задачу **управляемого жизненного цикла парка арендных активов**: от появления единицы до выбытия, с деньгами, сроками, доказательствами и ответственностью на каждом шаге.

Система уже работает как продукт — с реальными пользователями, опубликованными мобильными клиентами и рабочим логистическим контуром, — и одновременно оставляется платформой, спроектированной под рост: жёсткие границы владения, версионированные контракты и культура доказуемого качества делают расширение предсказуемым, а аудит — воспроизводимым.

Три тезиса, которые стоит запомнить:

1. **Каждый статус имеет одного хозяина** — поэтому цифры в системе сходятся.
2. **Каждый межсервисный шаг идемпотентен и версионизируем** — поэтому процессы не теряются при сбоях.
3. **Каждое изменение подтверждено прогонами и артефактами** — поэтому в систему можно вкладываться как в долгосрочный актив.

Приложение А: каталог сервисов

Сервис	Runtime	Ответственность
api-gateway-service	Spring, stateless	Публичные /auth/**, /api/**; edge-JWT; transport failures
auth-service	Spring, stateful	OIDC/OAuth2, пользователи, роли, клиенты, доступ к складам
warehouse-service	Spring, stateful	Идентичность и lifecycle склада, timezone, admission
asset-service	Spring, stateful	Кабины, оборудование, остатки, holds, leases, asset-факты
task-board-service	Spring, stateful	Очереди, workforce, назначения, worker stream
maintenance-service	Spring, stateful	Каталог, сметы, ремонт, приёмка, решения о списании
inventory-service	Spring, stateful	Сессии инвентаризации, находки, сверка, публикация
logistics-service	Spring, stateful	Запросы аренды, возвраты, отгрузки, перемещения, саги
media-service	Go, stateful	Метаданные медиа, загрузки, MinIO, приватный доступ
dossier-service	Spring, read model	Междоменная проекция активности кабины
analytics-service	Spring, read model	KPI и dashboard-проекции
assistant-service	Spring, stateful	Диалоги ассистента, tool calls
logistics planner	Python, отдельный контур	Карта, изохроны, слоты, маршруты (PostGIS + Valhalla)
platform	Java-библиотеки	Технические контракты, Spring-starter, architecture tests

Приложение В: источники истины документа

- `README.ru.md` — обзор продукта, границы, порядок источников истины
- `docs/project-knowledge/architecture.md` — авторитетная таблица владельцев
- `docs/project-knowledge/cabin-lifecycle.ru.md` — подтверждённая карта цикла бытовки
- `docs/project-knowledge/runtime-flows.md`, `domain-logic.md` — потоки и логика
- `rwms-work.md` — полный секвенциальный аудит (135 карточек)
- `problem.md` — непрерывный журнал проблем и исправлений с gate-прогонами
- `contracts/openapi/`, `contracts/events/` — канонические границы